EMSI — Développement Front-End

Séance 2 Next.js — TP : Server Actions, API Routes & Auth

Projet	TaskFlow Next — suite de la Séance 1 Next.js
Objectif	Server Actions (CRUD), API Routes (backend intégré), Middleware, Auth cookies
Stack	Next.js 14+ (App Router) + TypeScript + json-server (remplacé en partie 3)
Prérequis	Séance 1 Next.js terminée (routing, Server/Client Components, fetch SSR)

Partie 1 : Server Action — Ajouter un projet

1.1 Créer le fichier d'actions

Créez app/actions/projects.ts :

```
'use server';

import { revalidatePath } from 'next/cache';

export async function addProject(formData: FormData) {
  const name = formData.get('name') as string;
  const color = formData.get('color') as string;

  await fetch('http://localhost:4000/projects', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name, color }),
  });

  revalidatePath('/dashboard');
}
```

'use server' en haut du fichier = TOUTES les fonctions exportées s'exécutent sur le serveur. Pas dans le navigateur.

1.2 Créer le composant formulaire

Créez app/dashboard/AddProjectForm.tsx :

```
'use client';

import { useFormStatus } from 'react-dom';
import { addProject } from '../actions/projects';

function SubmitButton() {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending} style={{
      padding: '8px 16px', background: '#1B8C3E', color: 'white',
      border: 'none', borderRadius: 4, cursor: 'pointer'
    }}>
      {pending ? 'Création...' : '+ Nouveau projet'}
```

```
    </button>
  );
}

export default function AddProjectForm() {
  return (
    <form action={addProject} style={{ display: 'flex', gap: 8, marginBottom: 16 }}>
      <input name="name" placeholder="Nom du projet" required
        style={{ padding: 8, borderRadius: 4, border: '1px solid #ccc' }} />
      <input name="color" type="color" defaultValue="#3498db"
        style={{ width: 40, height: 36, border: 'none' }} />
      <SubmitButton />
    </form>
  );
}
```

useFormStatus() doit être dans un composant ENFANT du <form>, pas dans le même composant. C'est pourquoi SubmitButton est séparé.

1.3 Intégrer dans le Dashboard

Modifiez app/dashboard/page.tsx :

```
import AddProjectForm from './AddProjectForm';

export default async function DashboardPage() {
  const res = await fetch('http://localhost:4000/projects', { cache: 'no-store' });
  const projects = await res.json();

  return (
    <div style={{ padding: '2rem' }}>
      <h1>Dashboard</h1>
      <AddProjectForm />
      <ul>
        {projects.map((p: any) => (
          <li key={p.id}>{p.name}</li>
        ))}
      </ul>
    </div>
  );
}
```

Testez : ajoutez un projet. La page se rafraîchit automatiquement avec le nouveau projet.

➤ *Q1 : En React SPA, que fallait-il faire après un POST pour voir le nouveau projet ? (indice : setProjects). Ici ?*

Partie 2 : Server Actions — Renommer et supprimer

À VOUS : Ajoutez deux Server Actions dans app/actions/projects.ts :

2.1 renameProject

Specs :

1. Reçoit formData avec 'id' et 'newName'
2. Fait un PUT vers http://localhost:4000/projects/{id}
3. Appelle revalidatePath('/dashboard')

Le body du PUT doit contenir tous les champs : { name: newName, color }.

2.2 deleteProject

À VOUS : Créez la Server Action deleteProject :

Specs :

1. Reçoit formData avec 'id'
2. Fait un DELETE vers http://localhost:4000/projects/{id}
3. Appelle revalidatePath('/dashboard')

2.3 Boutons dans le Dashboard

Ajoutez les boutons rename/delete à côté de chaque projet :

```
// Dans la boucle projects.map() du Dashboard :
<li key={p.id} style={{ display: 'flex', gap: 8, alignItems: 'center', marginBottom: 8 }}>
  <span style={{ width: 12, height: 12, borderRadius: '50%', background: p.color, display:
'inline-block' }} />
  <a href={`/projects/${p.id}`}>{p.name}</a>

  <form action={deleteProject} style={{ display: 'inline' }}>
    <input type="hidden" name="id" value={p.id} />
    <button type="submit" style={{ background: 'none', border: 'none', cursor: 'pointer' }}>
      🗑️
    </button>
  </form>
</li>
```

➤ *Q3 : Le bouton supprimer est un <form> avec un <input type="hidden">. Pourquoi pas un onClick ?*

Le Dashboard est un Server Component. Pas de onClick possible ! Le formulaire est la façon native d'envoyer des données au serveur sans JavaScript.

Partie 3 : API Routes — Backend intégré

Au lieu d'appeler json-server, on crée nos propres endpoints API dans Next.js.

3.1 GET et POST

Créez app/api/projects/route.ts :

```
import { NextResponse } from 'next/server';
import fs from 'fs';
import path from 'path';

const DB_PATH = path.join(process.cwd(), 'db.json');

function readDB() {
  const data = JSON.parse(fs.readFileSync(DB_PATH, 'utf-8'));
  return data;
}

function writeDB(data: any) {
  fs.writeFileSync(DB_PATH, JSON.stringify(data, null, 2));
}

export async function GET() {
  const db = readDB();
  return NextResponse.json(db.projects);
}

export async function POST(request: Request) {
  const body = await request.json();
  const db = readDB();
  const newProject = {
    id: String(Date.now()),
    name: body.name,
    color: body.color,
  };
  db.projects.push(newProject);
  writeDB(db);
  return NextResponse.json(newProject, { status: 201 });
}
```

Plus besoin de json-server ! Next.js EST le serveur. Les API Routes lisent et écrivent directement dans db.json.

3.2 Mettre à jour les Server Actions

Modifiez app/actions/projects.ts et app/dashboard/page.tsx pour pointer vers votre API :

```
// Remplacez http://localhost:4000/projects par :
await fetch('http://localhost:3000/api/projects', { ... });

// Ou mieux, utilisez une URL relative :
await fetch(`${process.env.NEXT_PUBLIC_URL || 'http://localhost:3000'}/api/projects`, { ...
});
```

Arrêtez json-server. Tout passe par les API Routes maintenant.

➤ *Q4 : Testez http://localhost:3000/api/projects dans le navigateur. Que voyez-vous ?*

➤ *Q5 : Quelle est la différence entre une API Route et une Server Action ?*

3.3 API Route dynamique

À VOUS : Créez app/api/projects/[id]/route.ts avec PUT et DELETE.

Specs :

GET : retourner le projet par id (ou 404)

PUT : modifier le nom du projet

DELETE : supprimer le projet du tableau

Partie 4 : Auth avec cookies

4.1 Server Action Login

Créez app/actions/auth.ts :

```
'use server';

import { cookies } from 'next/headers';
import { redirect } from 'next/navigation';

export async function login(prevState: any, formData: FormData) {
  const email = formData.get('email') as string;
  const password = formData.get('password') as string;

  if (email !== 'admin@taskflow.com' || password !== 'password123') {
    return { error: 'Email ou mot de passe incorrect' };
  }

  const cookieStore = await cookies();
  cookieStore.set('session', JSON.stringify({
    email, name: 'Admin', role: 'admin'
  }), {
    httpOnly: true,
    secure: false,
    maxAge: 3600,
    path: '/',
  });

  redirect('/dashboard');
}

export async function logoutAction() {
  const cookieStore = await cookies();
  cookieStore.delete('session');
  redirect('/login');
}
```

httpOnly: true = le cookie est INVISIBLE par JavaScript. Aucun script ne peut faire document.cookie pour le voler. C'est la protection XSS ultime.

4.2 Modifier Login

Modifiez app/login/page.tsx :

```
'use client';

import { useActionState } from 'react';
import { login } from '../actions/auth';

export default function LoginPage() {
  const [state, formAction, pending] = useActionState(login, null);

  return (
    <div style={{ padding: '2rem', maxWidth: 400, margin: '0 auto' }}>
      <h1 style={{ color: '#1B8C3E' }}>TaskFlow</h1>
      <p>Connectez-vous pour continuer</p>

      {state?.error && <p style={{ color: 'red' }}>{state.error}</p>}

      <form action={formAction} style={{ display: 'flex', flexDirection: 'column', gap: 12
}}>
        <input name="email" type="email" placeholder="Email" required
          style={{ padding: 8, borderRadius: 4, border: '1px solid #ccc' }} />
        <input name="password" type="password" placeholder="Mot de passe" required
          style={{ padding: 8, borderRadius: 4, border: '1px solid #ccc' }} />
        <button type="submit" disabled={pending} style={{
          padding: 10, background: '#1B8C3E', color: 'white',
          border: 'none', borderRadius: 4, cursor: 'pointer'
        }}>
          {pending ? 'Connexion...' : 'Se connecter'}
        </button>
      </form>
    </div>
  );
}
```

➤ *Q6 : Comparez ce Login avec celui de React SPA. Combien de useState en moins ?*

➤ *Q7 : Après le login, ouvrez F12 > Application > Cookies. Voyez-vous le cookie 'session' ? Pouvez-vous le lire avec document.cookie dans la console ?*

Partie 5 : Middleware

Créez middleware.ts à la RACINE du projet (PAS dans app/) :

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const session = request.cookies.get('session');

  // Si pas de cookie et route protégée → redirect login
  if (!session) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  return NextResponse.next();
}

// Quelles routes protéger :
export const config = {
  matcher: ['/dashboard/:path*', '/projects/:path*'],
};
```

Testez : déconnectez-vous, puis tapez /dashboard dans l'URL.

➤ *Q8 : En React SPA, ProtectedRoute affichait brièvement le Dashboard avant de rediriger. Ici, que se passe-t-il ?*

La page ne se charge MÊME PAS. Le middleware intercepte AVANT que le serveur génère le HTML. Zéro flash, zéro fuite de contenu.

➤ *Q9 : Le middleware.ts est à la racine, pas dans app/. Pourquoi ?*

Partie 6 : Logout + Afficher le user

6.1 Bouton Logout dans le Header

Créez app/components/LogoutButton.tsx :

```
'use client';

import { logoutAction } from '../actions/auth';

export default function LogoutButton() {
  return (
    <form action={logoutAction}>
      <button type="submit" style={{
        background: 'none', border: '1px solid white', color: 'white',
        padding: '4px 12px', borderRadius: 4, cursor: 'pointer'
      }}>
        Déconnexion
      </button>
    </form>
  );
}
```

6.2 Afficher le user dans le layout

Modifiez app/layout.tsx pour lire le cookie :

```
import { cookies } from 'next/headers';
```

```
import LogoutButton from './components/LogoutButton';

export default async function RootLayout({ children }: { children: React.ReactNode }) {
  const cookieStore = await cookies();
  const session = cookieStore.get('session');
  const user = session ? JSON.parse(session.value) : null;

  return (
    <html lang="fr">
      <body>
        <header style={{ background: '#1B8C3E', color: 'white', padding: '1rem 2rem',
display: 'flex', justifyContent: 'space-between' }}>
          <h2 style={{ margin: 0 }}>TaskFlow</h2>
          <div style={{ display: 'flex', alignItems: 'center', gap: 12 }}>
            {user && <span>{user.name}</span>}
            {user && <LogoutButton />}
            {!user && <a href="/login" style={{ color: 'white' }}>Login</a>}
          </div>
        </header>
        <main>{children}</main>
      </body>
    </html>
  );
}
```

➤ *Q10 : Le layout est un Server Component. Il lit le cookie DIRECTEMENT avec cookies(). En React SPA, comment faisait-on ? (indice : useAuth(), Context, state)*

Partie 7 : Questions de réflexion

- ➢ *Q11 : Server Actions vs API Routes — lequel utiliseriez-vous pour un formulaire de création de projet ? Pour une app mobile qui consomme la même API ?*
- ➢ *Q12 : En React SPA, l'auth était : Context + useReducer + JWT mémoire + ProtectedRoute. En Next.js c'est : cookies + middleware. Quel avantage de sécurité ?*
- ➢ *Q13 : Si vous arrêtez json-server, les API Routes fonctionnent-elles toujours ? Pourquoi ?*
- ➢ *Q14 : Le cookie est HttpOnly. Un script XSS injecté dans la page peut-il le voler ?*

Récap : structure du projet

```
taskflow-next/
  app/
  ├── layout.tsx               // Header + cookies() + LogoutButton
  ├── page.tsx                 // accueil
  ├── login/
  │   └── page.tsx              // 'use client' + useActionState + login
  ├── dashboard/
  │   ├── page.tsx              // Server Component + fetch + AddProjectForm
  │   └── AddProjectForm.tsx    // 'use client' + form action={addProject}
  ├── projects/[id]/
  │   └── page.tsx              // Server Component + fetch par id
  ├── actions/
  │   ├── projects.ts           // 'use server' : add, rename, delete
  │   └── auth.ts               // 'use server' : login, logout
  ├── api/projects/
  │   ├── route.ts              // GET + POST
  │   └── [id]/route.ts         // GET + PUT + DELETE
  └── components/
      └── LogoutButton.tsx      // 'use client' + form action={logout}

  middleware.ts                 // protège /dashboard et /projects/*
  db.json                       // données (lu par les API Routes)
```